

LISTA DE EXERCÍCIOS 3.3

1. Levando em consideração o tempo de chaveamento de contexto, discorra sobre o impacto de *swapping*.

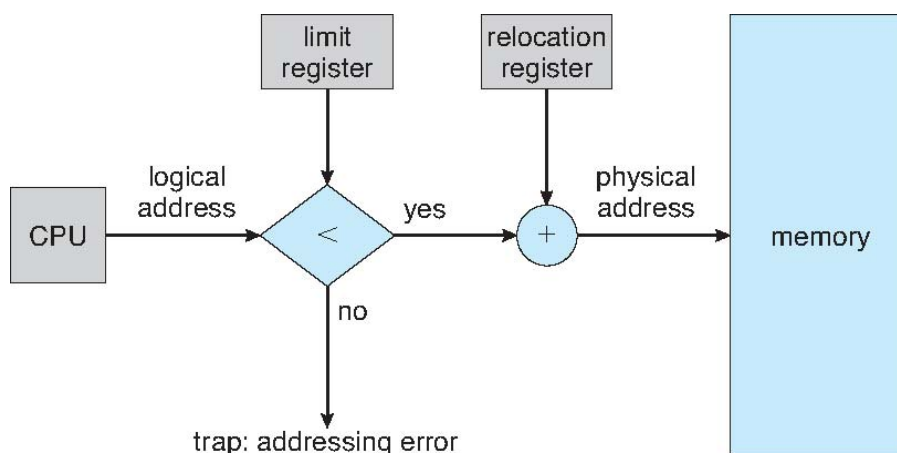
Resposta: Se os próximos processos a serem colocados na CPU não estão na memória, pode ser necessário remover um processo (*swap out*) e trazer um processo alvo (*swap in*), dependendo da disponibilidade de memória no sistema. Assim, em caso de *swapping*, o tempo de chaveamento de contexto pode ser muito alto. Por esse motivo, em sistemas modernos o *swap* só ocorre caso a memória livre esteja extremamente baixa. Outro ponto a ser observado refere-se a I/O pendente (i.e. processo inicia I/O e entra na fila de processos bloqueados). Nesse caso, é recomendável que o processo permaneça em memória, fixado de alguma forma, de modo a evitar *double buffering* (i.e. a perda do que foi lido em I/O e nova leitura devido ao *swap out* do processo).

2. Considere um sistema no qual um programa pode ser separado em duas partes: código e dados. A CPU sabe se irá acessar uma instrução (busca por instrução) ou dados (busca ou armazenamento de dados). Assim, dois pares de registradores base-limite são oferecidos: um para instruções e um para dados. O par de registradores base-limite para instrução está automaticamente em modo *read-only*, de modo que programas podem ser compartilhados entre usuários diferentes. Discuta as vantagens e desvantagens deste esquema.

Resposta: A maior vantagem deste esquema é que este é um mecanismo efetivo para compartilhamento de código e dados. Por exemplo, somente uma cópia de um editor ou de um compilador precisa ser mantida em memória, e este código pode ser compartilhado por todos os processos que precisem de acesso ao editor ou ao código do compilador. Outra vantagem é a proteção do código contra modificações errôneas. A única desvantagem é que o código e os dados devem ser separados, o que geralmente é unificado em um código gerado por compilador.

3. Qual é o papel dos registradores base e limite em alocação contígua?

Resposta: no momento em que a MMU deve traduzir um endereço lógico em físico, o registrador base (ou registrador de realocação), contém valor do menor endereço físico e o registrador limite contém o escopo de variação de endereços lógicos. Nesse contexto, cada endereço lógico deve ser menor do que o registrador limite. Em caso de erro de escopo do registrador lógico, ocorre uma *trap* por erro de endereçamento.



4. Considerando o modelo de alocação por partição múltipla, explique o que vem a ser fragmentação externa.

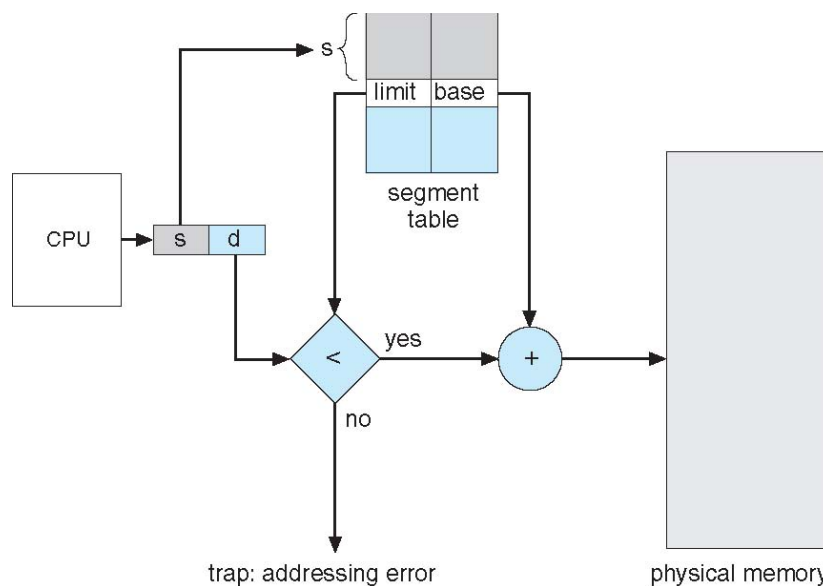
Resposta: em alocação por partição múltipla, devido ao tamanho da partição ser variável e a alocação de memória ocorrer quando um processo chega em uma posição que seja “grande o bastante para acomodá-lo”, com a alocação e liberação constante de partições, podem surgir buracos em memória não adjacentes (i.e. não contíguos), os quais somados poderiam acomodar um processo de tamanho m , mas isolados não são capazes de conter um processo que perfaz esse tamanho total.

5. Explique em que contexto ocorre fragmentação interna.

Resposta: considerando a alocação de partições de memória de tamanho fixo, a fragmentação interna ocorre devido ao não preenchimento total de uma partição de memória.

6. Como ocorre a tradução de endereços no caso de segmentação?

Resposta: para um dado segmento, a MMU verifica na tabela de segmentos inicialmente se o deslocamento a ser usado está dentro do escopo do processo (i.e. se é condizente com o valor do registrador limite). Em caso positivo, utiliza o endereço base para o cálculo do endereço físico resultante para referência em memória.



7. Descreva algum mecanismo pelo qual um segmento poderia pertencer ao espaço de endereçamento de dois processos diferentes.

Resposta: Já que tabelas de segmento são uma coleção de registradores base-limite, segmentos podem ser compartilhados quando entradas na tabela de segmento de duas tarefas diferentes apontam para a mesma localização física. As duas tabelas de segmento devem ter ponteiros base idênticos, e o número de segmento compartilhado deve ser o mesmo nos dois processos.